BLOCKSEC

Security Audit

Report for agent-router

Date: May 27, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Report Manifest

Item	Description
Client	baelor
Target	agent-router

Version History

Version	Date	Description
1.0	May 27, 2026	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-05-28 03:02:06+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository [1] of agent-router of baelor.

The project introduces `AgentRouter`, a permissionless entry contract for the Cobo tokenized gold ecosystem. This contract mediates between XAUT and XAUE by routing user mint and redemption operations through an underlying contract, `CoboFundToken`. Specifically, it allows any user to deposit XAUT and receive XAUE shares, as well as to redeem XAUE shares back to XAUT via an asynchronous redemption flow. Additionally, the contract implements a blacklist mechanism to block sanctioned or non-compliant users from participating in either minting or redemption operations.

Note this audit only focuses on the smart contracts in the following directories/files:

- contracts/AgentRouter.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
agent-router	`Version 1`	`24f57de712fb54ee0f5f5893573f6ef80753b20b`
	`Version 2`	`dd38f360b889bdff1ca1b26f9231d445ef31354d`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

[1] https://github.com/baelor-git/agent-router.git

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **two** potential security issues. Besides, we have **five** recommendations and **four** notes.

- Medium Risk: 2
- Recommendation: 5
- Note: 4

ID	Severity	Description	Category	Status
1	Medium	Circumvention of `blacklist` check	Security Issue	Confirmed
2	Medium	Lack of blacklist and pause validation	Security Issue	Confirmed
3	-	Rename field `navReqId` in struct `RouterRedemption`	Recommendation	Confirmed
4	-	Use explicit status for rejected claimable requests	Recommendation	Confirmed
5	-	Correct length of storage gaps	Recommendation	Fixed
6	-	Add rescue token functionality	Recommendation	Fixed
7	-	Implement slippage protection on functions `mint()` and `requestRedeem()`	Recommendation	Fixed
8	-	Potential centralization risks	Note	-
9	-	Proxy deployment and implementation binding should be atomic	Note	-
10	-	Token implementation assumption	Note	-
11	-	Assumptions on compliance responsibility	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Circumvention of `blacklist` check

Severity Medium

Status Confirmed

Introduced by `Version 1`

Description In the contract `AgentRouter`, the functions `mint()` and `requestRedeem()` perform a `blacklist` check to prevent sanctioned users from minting or redeeming. However, this protection is ineffective as any blocked address can circumvent the restrictions using an unlisted intermediary address.

```
125   /// @notice Pull XAUT from user, mint XAUE via fund, transfer XAUE to user.
126   function mint(uint256 xautAmount) external whenNotPaused nonReentrant {
127       if (blacklist[msg.sender]) revert Blacklisted();
128       if (xautAmount == 0) revert ZeroAmount();
129
130       asset.safeTransferFrom(msg.sender, address(this), xautAmount);
131       asset.safeIncreaseAllowance(address(fundToken), xautAmount);
```

```
132
133         uint256 xaueAmount = fundToken.mint(xautAmount);
134
135         asset.forceApprove(address(fundToken), 0);
136
137         fundToken.safeTransfer(msg.sender, xaueAmount);
138         emit MintRouted(msg.sender, xautAmount, xaueAmount);
139     }
```

Listing 2.1: contracts/AgentRouter.sol

```
141     /// @notice Pull XAUE from user, open redemption on fund, record `routerReqId`.
142     function requestRedeem(uint256 xaueAmount) external whenNotPaused nonReentrant returns (
            uint256 routerReqId) {
143         if (blacklist[msg.sender]) revert Blacklisted();
144         if (xaueAmount == 0) revert ZeroAmount();
145
146         fundToken.safeTransferFrom(msg.sender, address(this), xaueAmount);
147
148         uint256 navReqId = fundToken.requestRedemption(xaueAmount);
149
150         routerReqId = nextRouterReqId++;
151         routerRedemptions[routerReqId] = RouterRedemption({
152             id: routerReqId,
153             user: msg.sender,
154             navReqId: navReqId,
155             xaueAmount: xaueAmount,
156             requestedAt: block.timestamp,
157             xautClaimed: false,
158             rejectedSharesClaimed: false
159         });
160
161         emit RedemptionRequestedViaRouter(routerReqId, navReqId, msg.sender, xaueAmount);
162     }
```

Listing 2.2: contracts/AgentRouter.sol

Impact Malicious users can circumvent the `blacklist` check.

Suggestion Revise the code logic accordingly.

Feedback from the project The project states that this is a known limitation of the current design, and they accept it.

2.1.2 Lack of blacklist and pause validation

Severity Medium

Status Confirmed

Introduced by `Version 1`

Description In the contract `AgentRouter`, functions `mint()` and `requestRedeem()` check `blacklist[msg.sender]` and enforce `whenNotPaused` before execution. However, the functions `claimXaut()`

and `claimRejectedShares()` do not enforce these checks. As a result, users can still claim routed assets or shares after being blacklisted, or while the router is paused.

```
185   /// @notice After fund redemption is executed, transfer XAUT payout from Router to user.
186   function claimXaut(uint256 routerReqId) external nonReentrant {
187       RouterRedemption storage r = routerRedemptions[routerReqId];
188       if (r.user != msg.sender) revert NotRequestOwner();
189       if (r.xautClaimed) revert AlreadyClaimed();
190
191       (, , uint256 assetAmount, , , ICoboFundToken.RedemptionStatus st) = fundToken.redemptions(r
              .navReqId);
192       if (st != ICoboFundToken.RedemptionStatus.Executed) revert NotClaimable();
193       if (asset.balanceOf(address(this)) < assetAmount) revert InsufficientAsset();
194
195       r.xautClaimed = true;
196       asset.safeTransfer(msg.sender, assetAmount);
197       emit XautClaimed(routerReqId, msg.sender, assetAmount);
198   }
199
200   /// @notice After fund redemption is rejected, return XAUE shares from Router to user.
201   function claimRejectedShares(uint256 routerReqId) external nonReentrant {
202       RouterRedemption storage r = routerRedemptions[routerReqId];
203       if (r.user != msg.sender) revert NotRequestOwner();
204       if (r.rejectedSharesClaimed) revert AlreadyClaimed();
205
206       (, , , uint256 shareAmount, , ICoboFundToken.RedemptionStatus st) = fundToken.redemptions(r
              .navReqId);
207       if (st != ICoboFundToken.RedemptionStatus.Rejected) revert NotRejected();
208       if (fundToken.balanceOf(address(this)) < shareAmount) revert InsufficientShares();
209
210       r.rejectedSharesClaimed = true;
211       fundToken.safeTransfer(msg.sender, shareAmount);
212       emit RejectedSharesClaimed(routerReqId, msg.sender, shareAmount);
213   }
```

Listing 2.3: contracts/AgentRouter.sol

```
142   function requestRedeem(uint256 xaueAmount) external whenNotPaused nonReentrant returns (
          uint256 routerReqId) {
143       if (blacklist[msg.sender]) revert Blacklisted();
144       if (xaueAmount == 0) revert ZeroAmount();
145
146       fundToken.safeTransferFrom(msg.sender, address(this), xaueAmount);
147
148       uint256 navReqId = fundToken.requestRedemption(xaueAmount);
149
150       routerReqId = nextRouterReqId++;
151       routerRedemptions[routerReqId] = RouterRedemption({
152           id: routerReqId,
153           user: msg.sender,
154           navReqId: navReqId,
155           xaueAmount: xaueAmount,
156           requestedAt: block.timestamp,
157           xautClaimed: false,
```

```
158             rejectedSharesClaimed: false
159         });
160
161         emit RedemptionRequestedViaRouter(routerReqId, navReqId, msg.sender, xaueAmount);
162     }
```

Listing 2.4: contracts/AgentRouter.sol

Impact Users can still claim assets or shares after being blacklisted, or while the router is paused.

Suggestion Add blacklist and pause checks to the functions `claimXaut()` and `claimRejectedShares()`. Additionally, implement a sanction handling mechanism for blacklisted users with pending claims.

Feedback from the project The project states that this behavior is by design. Functions `mint()` and `requestRedeem()` enforce `whenNotPaused` and `blacklist[msg.sender]` to block new entry and redemption requests, while functions `claimXaut()` and `claimRejectedShares()` intentionally lack these checks, allowing users to claim approved redemptions even if subsequently blacklisted or while the router is paused.

2.2 Recommendation

2.2.1 Rename field `navReqId` in struct `RouterRedemption`

Status Confirmed

Introduced by `Version 1`

Description In the contract `AgentRouter`, the field `navReqId` in the struct `RouterRedemption` represents an underlying redemption request identifier rather than NAV-related data. The current naming may confuse developers and integrators. It is recommended to rename this field to a more descriptive identifier.

```
44    struct RouterRedemption {
45        uint256 id;
46        address user;
47        uint256 navReqId;
48        uint256 xaueAmount;
49        uint256 requestedAt;
50        bool xautClaimed;
51        bool rejectedSharesClaimed;
52    }
```

Listing 2.5: contracts/AgentRouter.sol

Suggestion Rename the field `navReqId` in the struct `RouterRedemption` to a more descriptive identifier, such as `redemptionReqId`.

2.2.2 Use explicit status for rejected claimable requests

Status Confirmed

Introduced by Version 1

Description In the contract AgentRouter, the function getRouterRedemptionStatus() returns Rejected when the underlying redemption is rejected, but the shares are still claimable. This makes the status semantics inconsistent with the status Claimable, which is used for executed requests whose shares can also be claimed. It is recommended to introduce a more explicit status name to distinguish the rejected-but-claimable state.

```
164   function getRouterRedemptionStatus(uint256 routerReqId) public view returns (
          RouterRedemptionStatus) {
165       RouterRedemption memory r = routerRedemptions[routerReqId];
166       if (r.user == address(0)) revert UnknownRequest();
167
168       (, , , , , ICoboFundToken.RedemptionStatus st) = fundToken.redemptions(r.navReqId);
169
170       if (st == ICoboFundToken.RedemptionStatus.Pending) {
171           return RouterRedemptionStatus.Pending;
172       }
173       if (st == ICoboFundToken.RedemptionStatus.Executed) {
174           return r.xautClaimed ? RouterRedemptionStatus.Claimed : RouterRedemptionStatus.
                  Claimable;
175       }
176       return r.rejectedSharesClaimed ? RouterRedemptionStatus.Claimed : RouterRedemptionStatus.
              Rejected;
177   }
```

Listing 2.6: contracts/AgentRouter.sol

Suggestion Use an explicit claimable status for rejected requests.

2.2.3 Correct length of storage gaps

Status Fixed in Version 2

Introduced by Version 1

Description In upgradeable contracts, the array __gap usually has a length of (50 - occupied slot) to reserve space for future upgrades. The contract AgentRouter has 5 slots occupied by the storage variables. However, the array __gap is declared with a length of 50, which does not align with the standard upgradeable storage pattern.

```
215   uint256[50] private __gap;
```

Listing 2.7: contracts/AgentRouter.sol

Suggestion Adjust the __gap length to conform to the standard 50-slot layout.

2.2.4 Add rescue token functionality

Status Fixed in Version 2

Introduced by Version 1

Description The contract `AgentRouter` has no function to rescue ERC20 tokens, which might be mistakenly sent to the contract. It is recommended to add a rescue function in the contract `AgentRouter`.

Suggestion Add rescue token functionality in the contract `AgentRouter`.

2.2.5 Implement slippage protection on functions `mint()` and `requestRedeem()`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `AgentRouter`, functions `mint()` and `requestRedeem()` allow users to deposit assets or submit redemption requests. However, neither function allows users to specify a minimum acceptable output as slippage protection. Both operations use the oracle price to determine the execution amount, while the oracle price may be updated at an uncertain time. As a result, if the price changes before the transaction is confirmed, users may receive a lower value than expected.

```
125   /// @notice Pull XAUT from user, mint XAUE via fund, transfer XAUE to user.
126   function mint(uint256 xautAmount) external whenNotPaused nonReentrant {
127       if (blacklist[msg.sender]) revert Blacklisted();
128       if (xautAmount == 0) revert ZeroAmount();
129
130       asset.safeTransferFrom(msg.sender, address(this), xautAmount);
131       asset.safeIncreaseAllowance(address(fundToken), xautAmount);
132
133       uint256 xaueAmount = fundToken.mint(xautAmount);
134
135       asset.forceApprove(address(fundToken), 0);
136
137       fundToken.safeTransfer(msg.sender, xaueAmount);
138       emit MintRouted(msg.sender, xautAmount, xaueAmount);
139   }
140
141   /// @notice Pull XAUE from user, open redemption on fund, record `routerReqId`.
142   function requestRedeem(uint256 xaueAmount) external whenNotPaused nonReentrant returns (
          uint256 routerReqId) {
143       if (blacklist[msg.sender]) revert Blacklisted();
144       if (xaueAmount == 0) revert ZeroAmount();
145
146       fundToken.safeTransferFrom(msg.sender, address(this), xaueAmount);
147
148       uint256 navReqId = fundToken.requestRedemption(xaueAmount);
149
150       routerReqId = nextRouterReqId++;
151       routerRedemptions[routerReqId] = RouterRedemption({
152           id: routerReqId,
153           user: msg.sender,
154           navReqId: navReqId,
155           xaueAmount: xaueAmount,
156           requestedAt: block.timestamp,
157           xautClaimed: false,
```

```
158             rejectedSharesClaimed: false
159         });
160
161         emit RedemptionRequestedViaRouter(routerReqId, navReqId, msg.sender, xaueAmount);
162     }
```

Listing 2.8: contracts/AgentRouter.sol

Suggestion Add a slippage parameter and the corresponding validation to these functions.

2.3 Note

2.3.1 Potential centralization risks

Introduced by Version 2

Description In this project, several privileged roles in the contract `AgentRouter` (e.g., `DEFAULT_ADMIN_ROLE`, `BLACKLIST_ADMIN_ROLE`, and `PAUSER_ROLE`) are granted the ability to conduct sensitive operations. The `DEFAULT_ADMIN_ROLE` can upgrade the contract implementation via UUPS, grant or revoke any role, effectively rewrite all logic, and use function `rescueERC20()` to withdraw any tokens except `asset` and `fundToken`. The `PAUSER_ROLE` can halt all new `mint()` and `requestRedeem()` operations indefinitely, while the `BLACKLIST_ADMIN_ROLE` can selectively block any address from participating in minting or redemption. These privileges introduce potential centralization risks. If the private keys of these privileged accounts are lost or maliciously exploited, user operations may be blocked, improperly restricted, or redirected through a malicious upgraded implementation.

2.3.2 Proxy deployment and implementation binding should be atomic

Introduced by Version 1

Description To prevent potential front-running attacks, it is recommended that proxy deployment and implementation binding be executed atomically within a single transaction. If these operations are performed separately, malicious actors could exploit the time window between deployment and binding to front-run the implementation binding transaction. An attacker could potentially bind their own malicious implementation to the newly deployed proxy contract, gaining unauthorized control over the protocol's upgrade mechanism and user funds. This race condition poses significant security risks, including complete protocol compromise, fund theft, and unauthorized access to privileged functions.

2.3.3 Token implementation assumption

Introduced by Version 1

Description In contract `AgentRouter`, the minting and redemption flows rely on the specific implementations of `asset` and `fundToken`. This audit operates under the following conditions:

1. Neither token is fee-on-transfer nor rebasing tokens, and their transfer operations impose no constraints on the recipient.

2. When a fund redemption request is rejected, the full amount is refunded without fees.

Feedback from the project The project confirms that the tokens satisfy these conditions. The `asset` represents `XAUT`, while the `fundToken` represents the `CoboFundToken`, which was previously audited [1].

2.3.4 Assumptions on compliance responsibility

Introduced by `Version 1`

Description In the project, users initiate mint or redeem requests through contract `AgentRouter`, which then invokes the contract `CoboFundToken`. The contract `CoboFundToken` applies a whitelist check on `msg.sender` (i.e., `AgentRouter`), while the contract `AgentRouter` implements a blacklist check on end users. Additionally, `CoboFundToken` does not consider the `AgentRouter`'s blacklist when approving redemption requests.

Therefore, the integration relies on the assumption that contract `AgentRouter` enforces the blacklist correctly. Otherwise, non-compliant users may be able to participate in minting or redemption flows.

Feedback from the project The project states that this behaviour is by design. The contract `AgentRouter` only enforces blacklist checks on end users, while the redemption approval mechanism in contract `CoboFundToken` is determined based on its own state.

[1] https://github.com/blocksecteam/audit-reports/blob/main/solidity/blocksec_cobo_cobotokenization_signed_20260401.pdf


BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS